Chapter 5

PPO — Proximal Policy Optimization

5.1 Motivation and History

Problem: Vanilla policy gradient updates have no constraint on step size. A single unlucky batch can push the policy into a region where it generates garbage → garbage gets low rewards → next gradient makes things worse → unrecoverable collapse.

Solution history:

1. **TRPO** [317] (2015): Constrain KL divergence between old and new policy. Works perfectly but requires expensive second-order optimization (Fisher information matrix, conjugate gradients).
2. **PPO** (2017) [319]: Achieve similar stability with a simple first-order clipped objective. 10× simpler to implement, works almost as well, scales to distributed training trivially.

5.2 The Clipped Objective

The core innovation of PPO is a clipped surrogate objective that prevents destructively large policy updates while remaining simple to implement.

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]} \tag{5.1}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio.

Clipping Intuition — The Key Insight

The min operator creates a **pessimistic bound**:

- **Good action** ($\hat{A} > 0$): We want to increase its probability. The surrogate $r\hat{A}$ grows as r increases. But clip caps benefit at $r = 1 + \epsilon$. *"Don't get greedy on one good example."*
- **Bad action** ($\hat{A} < 0$): We want to decrease its probability. $r\hat{A}$ improves as r decreases. But clip caps benefit at $r = 1 - \epsilon$. *"Don't forget too aggressively based on one bad example."*

Net effect: policy changes by at most ±20% per update step. Prevents both catastrophic collapse and overconfident specialization.

5.3 Full PPO Loss

$$L = L^{\text{CLIP}} - c_1 \underbrace{(V_\theta(s_t) - V_t^{\text{target}})^2}_{\text{value loss}} + c_2 \underbrace{H[\pi_\theta(\cdot|s_t)]}_{\text{entropy bonus}} \tag{5.2}$$

- **Value loss** ($c_1 = 0.1$): Trains the critic to predict returns. Also clipped for stability.

- **Entropy bonus** ($c_2 = 0.01$): Prevents premature convergence to deterministic policy. Critical for exploration.

5.4 Derivation of the PPO Gradient and Update Rule

This section traces the mathematical path from the RL objective to the PPO update rule, showing *why* the clipped surrogate works.

5.4.1 Step 1: The RL Objective

The goal is to maximize expected cumulative reward under the policy:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} r_t \right] \tag{5.3}$$

5.4.2 Step 2: Policy Gradient Theorem

The gradient of $J(\theta)$ with respect to policy parameters:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t \right]} \tag{5.4}$$

where $\hat{A}_t$ is the advantage function (how much better action a_t was compared to the average action in state s_t). This replaces the full return with the advantage to reduce variance.

5.4.3 Step 3: Importance Sampling for Off-Policy Data

PPO collects data using $\pi_{\theta_{\text{old}}}$ but updates π_θ. To correct for this distribution mismatch, apply importance sampling:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t \right] \tag{5.5}$$

Define the probability ratio $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$. Using the identity $\nabla_\theta \log f = \frac{\nabla_\theta f}{f}$, we get:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_{\theta_{\text{old}}}} \left[\nabla_\theta \, r_t(\theta) \cdot \hat{A}_t \right] \tag{5.6}$$

This means maximizing the **surrogate objective**:

$$L^{\text{CPI}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \cdot \hat{A}_t \right] \tag{5.7}$$

5.4.4 Step 4: The Problem with Unconstrained Surrogates

L^{CPI} is a valid objective, but without constraints, a single gradient step can push $r_t(\theta)$ far from 1.0, causing:

- Importance weights become extreme → high variance
- Policy enters untested regions → reward model gives unreliable scores
- Catastrophic collapse: policy generates garbage, can't recover

TRPO solution: Constrain $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \leq \delta$. Requires second-order methods (expensive).

5.4.5 Step 5: PPO's Clipped Surrogate (First-Order Approximation)

PPO replaces the hard KL constraint with a **clipped objective** that achieves similar behavior using only first-order gradients:

$$\boxed{L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\right]} \tag{5.8}$$

Derivation of the gradient:
Let $L_t = \min(r_t\hat{A}_t,\ \bar{r}_t\hat{A}_t)$ where $\bar{r}_t = \text{clip}(r_t, 1-\epsilon, 1+\epsilon)$.

$$\nabla_\theta L_t = \begin{cases} \nabla_\theta r_t(\theta)\cdot\hat{A}_t & \text{if } r_t\hat{A}_t < \bar{r}_t\hat{A}_t \text{ (unclipped term is smaller)} \\ 0 & \text{if } r_t\hat{A}_t \geq \bar{r}_t\hat{A}_t \text{ (clipped term is smaller, gradient = 0)} \end{cases} \tag{5.9}$$

Expanding the conditions:

- **When** $\hat{A}_t > 0$ **and** $r_t < 1+\epsilon$: Gradient flows normally — policy is encouraged to increase $\pi_\theta(a_t|s_t)$.
- **When** $\hat{A}_t > 0$ **and** $r_t \geq 1+\epsilon$: Gradient is **zero** — policy has already increased enough, stop pushing.
- **When** $\hat{A}_t < 0$ **and** $r_t > 1-\epsilon$: Gradient flows normally — policy is encouraged to decrease $\pi_\theta(a_t|s_t)$.
- **When** $\hat{A}_t < 0$ **and** $r_t \leq 1-\epsilon$: Gradient is **zero** — policy has already decreased enough, stop pushing.

5.4.6 Step 6: The Complete PPO Update Rule

Combining the clipped policy loss, value loss, and entropy bonus:

$$\boxed{\theta_{k+1} = \theta_k + \alpha\cdot\nabla_\theta\left[L^{\text{CLIP}}(\theta) - c_1 L^{\text{VF}}(\theta) + c_2 H[\pi_\theta]\right]} \tag{5.10}$$

where:

$$L^{\text{VF}}(\theta) = \left(V_\theta(s_t) - V_t^{\text{target}}\right)^2 \qquad \text{(value function regression loss)} \tag{5.11}$$

$$H[\pi_\theta] = -\sum_a \pi_\theta(a|s_t)\log\pi_\theta(a|s_t) \qquad \text{(entropy of the policy)} \tag{5.12}$$

Summary: Why This Works

1. **Policy gradient theorem** gives us the direction to improve the policy.
2. **Importance sampling** lets us reuse data from $\pi_{\theta_{\text{old}}}$ across multiple epochs.
3. **Clipping** prevents the importance weights from becoming extreme, keeping updates safe.
4. **The min operator** ensures we always take the more conservative of (clipped, unclipped) — a pessimistic lower bound on improvement.
5. **Result**: Monotonic improvement with probability 1, using only first-order gradients. No Hessians, no conjugate gradients, no line searches.

5.5 Rollout Buffer and Rollouts

In PPO, data management relies on a specialized, short-term storage system known as a **Rollout Buffer**. Unlike off-policy algorithms (DQN) that store experiences indefinitely in a replay buffer, PPO requires an ephemeral structure to satisfy its on-policy mathematical constraints.

5.5.1 What is a Rollout?

A **rollout** (trajectory) is a sequence of interactions generated by the agent running its current policy in the environment:

- **The process**: The agent observes a state, selects an action, receives a reward, and moves to the next state. It repeats for a fixed number of steps or until the episode ends.
- **In LLMs/RLHF**: A rollout consists of taking a prompt from a dataset and letting the language model generate a complete sequence of tokens token-by-token until an end-of-text marker is hit. Each token is one "step."

5.5.2 The Rollout Buffer

The rollout buffer temporarily stores all data collected during the rollout phase. For every generated token/step, it records:

$$\boxed{\mathcal{B} = \{(s_t,\ a_t,\ \log \pi_{\theta_{\text{old}}}(a_t|s_t),\ r_t,\ V(s_t))\}_{t=1}^{T}} \tag{5.13}$$

- s_t, a_t, r_t: State, action taken, and reward at step t.
- $\log \pi_{\theta_{\text{old}}}(a_t|s_t)$: Log-probability of taking that action under the exact policy that generated it (needed for ratio computation).
- $V(s_t)$: Value function's baseline prediction (needed for GAE advantage computation).

5.5.3 The Rollout Buffer Lifecycle

The buffer operates in a strict three-phase clockwork cycle:

1. **Collect**: The active policy interacts with the environment to fill the buffer with fresh trajectories (for a 70B model with batch=128, max_tokens=512: up to 65K token-level transitions per rollout).
2. **Train**: Compute GAE advantages across trajectories. Run K epochs (typically 3–10) of mini-batch gradient descent to update policy weights using the clipped objective.
3. **Purge**: The entire buffer is **completely wiped clean**. Because PPO is on-policy, data generated by the old policy cannot be safely reused for the next update cycle — the ratio $r_t(\theta)$ would become stale and the clipping guarantees would break.

Rollout Buffer vs Replay Buffer

Replay Buffer (DQN, SAC): Off-policy. Stores millions of transitions indefinitely. Random sampling. Data reused across many updates.
Rollout Buffer (PPO, GRPO): On-policy. Stores one batch of trajectories. Used for a few epochs, then discarded entirely. Fresh data required every cycle.
This is why PPO requires continuous generation — the buffer is emptied after every update, demanding fresh rollouts. This makes the generation bottleneck (60–70% of wall-clock time) particularly painful.

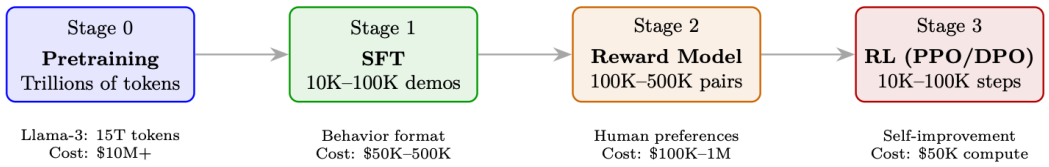
vLLM in RLHF Context

In RLHF training, vLLM is used for the **generation phase** (60–70% of wall-clock time). The policy model generates rollouts that are then scored by the reward model. Key benefits:

- **Batched generation**: Generate 256+ responses in parallel across prompts.
- **Memory efficiency**: Fit more concurrent generations → higher GPU utilization during the generation bottleneck.

- **Prefix sharing**: When generating $N = 8$ responses per prompt (GRPO), the prompt KV is computed once and shared across all 8 — no redundant prefill.
- **Integration**: Frameworks like OpenRLHF and TRL use vLLM as the generation backend, separating generation workers (vLLM) from training workers (DeepSpeed/FSDP).

5.6 PPO for RLHF: The Full Loop



Concrete PPO Step for a 70B Chat Model

Setup: Batch of 128 prompts, Llama-3-70B policy, 512 max tokens.
Step 1 — Generate: Sample 128 responses (temperature=0.7, top-p=0.9). This takes 60% of time.
Step 2 — Score: Reward model scores each (prompt, response) pair. Range: 0.2–0.95.
Step 3 — KL: Compute per-token KL: $\text{KL}_t = \log \pi_\theta(y_t|y_{<t}) - \log \pi_{\text{ref}}(y_t|y_{<t})$. Mean KL across tokens: typically 3–8.
Step 4 — Final reward: $R = r_{\text{RM}} - 0.05 \times \text{mean\_KL}$ (only at last token).
Step 5 — GAE: Compute $\hat{A}_t$ for each token position using value head predictions. Whiten advantages (zero mean, unit variance).
Step 6 — Update: 4 epochs of SGD on mini-batches of 16. Clip ratio $\epsilon = 0.2$. Gradient norm clipping at 1.0.
Result: Policy improves by ∼0.005 win-rate per step. After 10K steps: 5–10% absolute improvement over SFT.

Tokenization Pitfalls in RL for LLMs

When computing per-token KL penalties and advantages, remember that tokenization determines what a "step" is. A single conceptual action (e.g., outputting "2024") might span 1–4 tokens depending on the tokenizer. This creates subtle issues:

- **KL accounting**: Per-token KL sums to different totals for the same semantic content tokenized differently (e.g., rare words split into more subwords get higher total KL penalty).
- **Credit assignment**: GAE assigns advantage per token position—but semantic "decisions" often span multiple tokens. The model only truly "decides" at the first token of a word; subsequent subword tokens are largely deterministic.
- **Reward placement**: Placing reward only at the final token means all preceding tokens must propagate credit backward through GAE—longer responses suffer from more diluted signal.

Mitigation: Some systems normalize KL by sequence length, use word-level reward shaping, or apply reward at semantic boundaries rather than the final token.

5.7 Detailed Mechanics: Logits and Policy Updates

PPO manages two distinct parameter states in memory, which share the same neural network topology but hold different weight values during optimization:

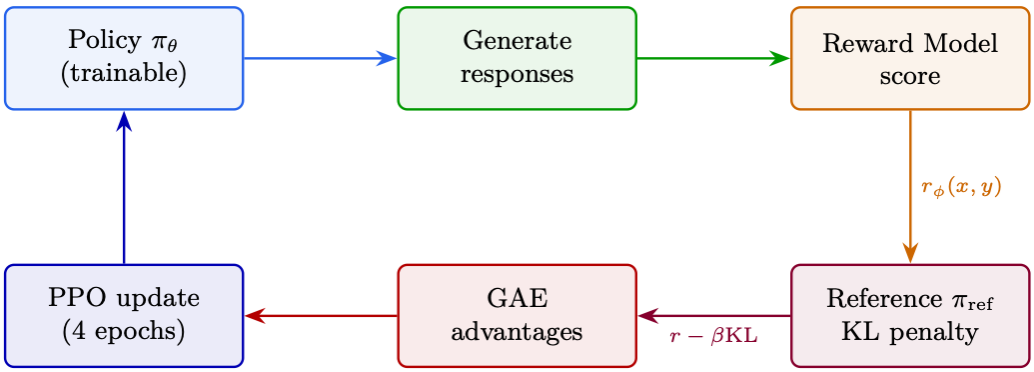


Figure 5.1: PPO end-to-end: from prompt batch through generation, reward scoring, KL computation, advantage estimation, to clipped policy update. The feedback loop shows the updated policy being used for the next generation step.

Core Architecture: Two Networks

1. **The Policy Network (π_θ):** The active, live network parameterized by weights θ. Continuously updated via backpropagation during optimization.
2. **The Old Policy Network ($\pi_{\theta_{\text{old}}}$):** A frozen snapshot parameterized by weights θ_{old}. Acts as a static anchor during a single optimization cycle to prevent the policy from shifting too drastically.

5.7.1 Phase 1: Rollout (Data Collection)

During data collection, the agent interacts with the environment for T steps. At each time-step t:

1. The environment yields the current state/observation s_t (for LLMs: prompt + tokens generated so far).
2. State s_t is passed through the current network snapshot (θ_{old}).
3. The network outputs raw unnormalized values — **logits** z_{old} — a vector of size $|V|$ (vocabulary size 32K–128K).
4. Probabilities are computed via Softmax:

$$P(a \mid s_t) = \text{Softmax}(z_{\text{old}}) = \frac{\exp(z_{\text{old},a})}{\sum_{j=1}^{|V|} \exp(z_{\text{old},j})} \tag{5.14}$$

5. An action a_t (next token) is sampled from $P(a \mid s_t)$, and the transition tuple $\langle s_t, a_t, r_t, s_{t+1} \rangle$ along with $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ is stored in the rollout buffer.

Why Store Log-Probabilities?

Storing $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ as a scalar during rollout avoids re-running the frozen network during optimization. This saves one full forward pass per mini-batch — significant for 70B models.

5.7.2 Phase 2: Optimization Loop (Mini-Batch Updates)

Once the rollout buffer is full, PPO runs K epochs (typically 3–10) over mini-batches. For every gradient step, logits are generated for both policies using the stored state s_t:

Old Policy Evaluation (frozen):

$$z_{\text{old}} = f(s_t; \theta_{\text{old}}) \quad \longrightarrow \quad \log \pi_{\theta_{\text{old}}}(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{old}})[a_t] \tag{5.15}$$

Implementation shortcut: reuse the stored scalar from rollout instead of re-computing.

Live Policy Evaluation (updating):

$$z_{\text{new}} = f(s_t;\theta) \quad \longrightarrow \quad \log \pi_\theta(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{new}})[a_t] \tag{5.16}$$

Because θ updates after every mini-batch gradient step, z_{new} changes continuously throughout the optimization loop, whereas z_{old} remains perfectly static.

5.7.3 From Logits to Probability Ratio

The core PPO ratio measures how much more or less likely an action is under the new policy vs the old:

$$\boxed{r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}} \tag{5.17}$$

To avoid catastrophic numerical underflow/overflow from dividing raw probabilities, the calculation is performed in **log-space**:

$$\log \pi_\theta(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{new}})[a_t] \tag{5.18}$$

$$\log \pi_{\theta_{\text{old}}}(a_t \mid s_t) = \text{LogSoftmax}(z_{\text{old}})[a_t] \tag{5.19}$$

The ratio is recovered via exponentiation of the difference:

$$\boxed{r_t(\theta) = \exp(\log \pi_\theta(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t))} \tag{5.20}$$

This ratio is injected into the PPO clipping objective:

$$\boxed{\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\ 1{-}\epsilon,\ 1{+}\epsilon)\ \hat{A}_t \Big) \right]} \tag{5.21}$$

How Clipping Works

- If $\hat{A}_t > 0$ (good action): ratio is clipped at $1 + \epsilon$ — cannot over-exploit good actions.
- If $\hat{A}_t < 0$ (bad action): ratio is clipped at $1 - \epsilon$ — cannot over-penalize bad actions.
- The $\min(\cdot)$ ensures we always take the more conservative estimate.

Result: monotonic improvement within a trust region — no catastrophic collapses.

5.7.4 The PPO Weight Lifecycle

Table 5.1: Evolution of θ and θ_{old} across PPO training phases.

Phase	**Live θ**	**Old θ_{old}**	**Ratio $r_t(\theta)$**
1. Rollout Start	Active copy	Same active copy	Always 1.0 (by identity)
2. Batch Step 1	Computes gradients	Frozen	1.0 (initial step)
3. Batch Step N	Modifying ($\theta \neq \theta_{\text{old}}$)	Frozen	Deviates from 1.0 (e.g., 1.06, 0.94)
4. Clipping Active	Bounded by ϵ	Frozen	Trapped at bound ($1 \pm \epsilon$)
5. Optimization End	Highly optimized	Discarded	N/A
6. Next Cycle	$\theta \rightarrow \theta_{\text{old}}$	Receives fresh θ	Resets back to 1.0

5.7.5 Continuous Action Spaces Extension

For continuous action spaces (not typical for LLMs, but important for robotics RL), the network outputs distribution parameters instead of discrete logits:

- Predicted mean vector μ

- Predicted standard deviation vector σ

Log-probabilities are computed via the Gaussian log-PDF:

$$\boxed{\log \pi(a_t \mid s_t) = -\frac{1}{2}\left(\frac{a_t - \mu}{\sigma}\right)^2 - \log(\sigma) - \frac{1}{2}\log(2\pi)} \tag{5.22}$$

The ratio $r_t(\theta) = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$ is then computed identically and fed into the same clipping objective.

5.8 TRL Implementation

The HuggingFace TRL library [376] provides production-ready implementations of all major RL methods for LLMs.

```
from trl import PPOConfig, PPOTrainer, AutoModelForCausalLMWithValueHead
from transformers import AutoTokenizer
from peft import LoraConfig

# Model setup
model = AutoModelForCausalLMWithValueHead.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    torch_dtype=torch.bfloat16, device_map="auto",
    peft_config=LoraConfig(r=64, lora_alpha=16, target_modules=["q_proj","v_proj",
    "k_proj","o_proj"])
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

# PPO config with all critical hyperparameters
ppo_config = PPOConfig(
    learning_rate=1.5e-6,        # Low LR for stability
    batch_size=128,              # Prompts per step
    mini_batch_size=16,          # Gradient accumulation unit
    ppo_epochs=4,                # Epochs per batch (reuse data)
    gamma=1.0,                   # No discounting (single turn)
    lam=0.95,                    # GAE lambda
    cliprange=0.2,               # PPO epsilon
    cliprange_value=0.2,         # Value function clipping
    vf_coef=0.1,                 # Value loss coefficient
    init_kl_coef=0.05,           # Initial KL penalty
    target_kl=6.0,               # Adaptive KL target
    whiten_rewards=True,         # Normalize advantages
    gradient_accumulation_steps=4,
    max_grad_norm=1.0,
)

ppo_trainer = PPOTrainer(config=ppo_config, model=model, tokenizer=tokenizer,
    dataset=prompt_dataset, data_collator=collator)

# Training loop
for batch in ppo_trainer.dataloader:
    # 1. Generate responses
    query_tensors = batch["input_ids"]
    response_tensors = ppo_trainer.generate(
        query_tensors, max_new_tokens=512, temperature=0.7, top_p=0.9, do_sample=
    True
    )
    # 2. Score with reward model
    texts = [tokenizer.decode(r, skip_special_tokens=True) for r in
    response_tensors]
    rewards = [torch.tensor(reward_model.score(q, r)) for q, r in zip(batch["query
    "], texts)]
    # 3. PPO update (handles KL, GAE, clipping internally)
```

```
    stats = ppo_trainer.step(query_tensors, response_tensors, rewards)
    # Monitor: stats["ppo/mean_scores"], stats["ppo/policy/approx_kl"]
```

5.9 Critical Hyperparameters

Parameter	Typical	Effect of Getting It Wrong
`cliprange`	0.2	Too low: no learning. Too high: instability.
`init_kl_coef`	0.01–0.1	Too low: reward hacking. Too high: stuck at SFT.
`target_kl`	4–8	Adaptive controller target. Lower = conservative.
`ppo_epochs`	4	Too many: overfits to batch. Too few: wastes gen compute.
`learning_rate`	$1{-}5 \times 10^{-6}$	Too high: catastrophic forgetting.
`batch_size`	64–256	Larger = smoother gradients, more gen compute.
`temperature`	0.7–1.0	Lower: less exploration. Higher: noisier advantages.